

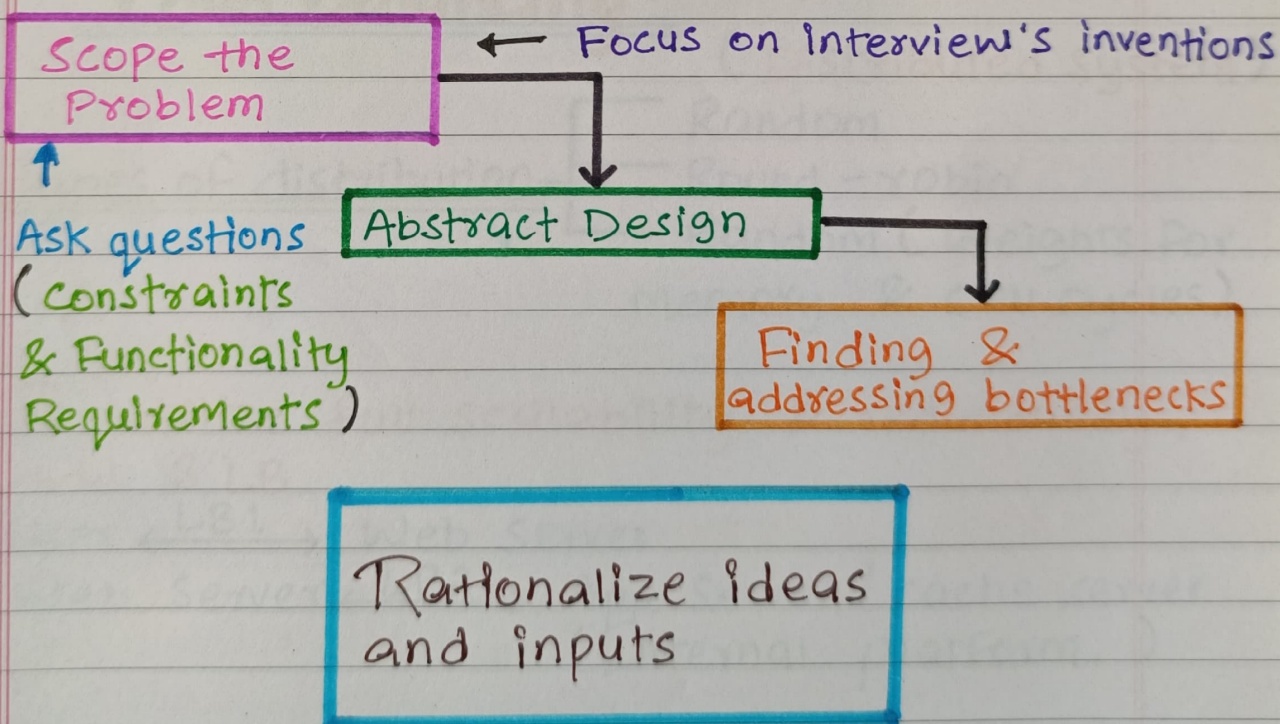
SYSTEM DESIGN

System Design Basics :

- 1). Try to break the problem into simpler modules.
(Top down approach)

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

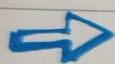
- 2). Talk about the trade-offs.
(No solution is perfect)
Calculated the impact on system based on all the constraints and the end test cases.



- Architectural Pieces / resources available
- How these resources work together.
- Utilization & Trade offs.

- Consistent Hashing
- CAP Theorem
- Load balancing
- Queues
- Caching
- Replication
- SQL vs NO-SQL
- Indexes
- Proxies
- Data Partitioning.

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).



Load Balancing

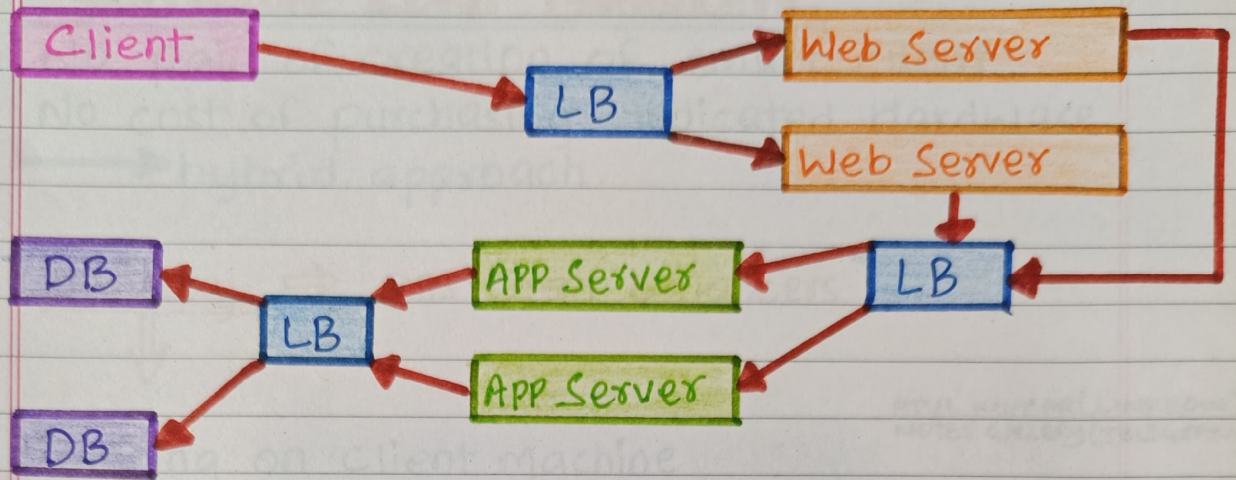
(Distributed System)

Types of distribution — { Random
Round-robin
Random (Weights for
memory & CPU cycles)

To utilize full scalability & redundancy,
add 3 LB.

- 1). User $\xleftarrow{\text{LB1}}$ Web server
- 2). Web server $\xleftarrow{\text{LB2}}$ App server / cache server
(Internal platform.)
3. Internal platform $\xleftarrow{\text{LB3}}$ DB.





ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).



Smart Clients

Take a pool of services hosts & balance load.

- detect hosts that are not responsible.
- recovered hosts.
- addition of new hosts.

Load balancing functionality to DB (Cache Services)

★ Attractive solution for developers.
(Small scale system)

As System grows → LBs (Standalone Servers).



Hardware load Balancers:

Expensive but high performance.

e.g. Citrix Netscaler

Not trivial to Configure

Large companies tend to avoid this Config. Or use it as 1st point of contact to their system to serve user request & Intra network uses smart clients/ hybrid solution → (Next page) for load balancing traffic.

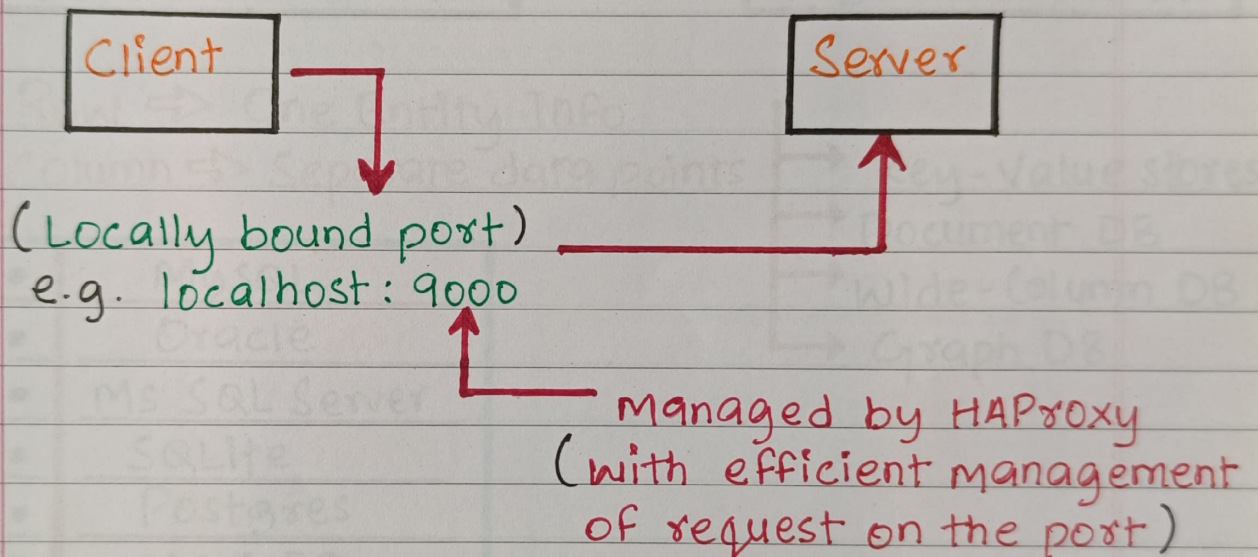
⇒ Software Load Balancers.

- No pain of creation of smart client
- No cost of purchasing dedicated Hardware.
- hybrid approach.

HAProxy ⇒ OSS Load balancers

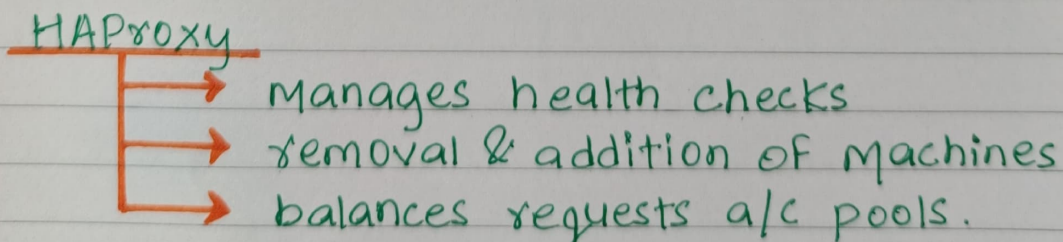


1]. Running on client machine



2]. Running on intermediate server:

Proxies running between different server side components



World of Database

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

SQL VS NO SQL

Relational Database

- Structured
- Predefined schema
- Data in rows & column

Non-relational Database

- Unstructured
- distributed
- dynamic schema

Row $\Rightarrow$ One Entity Info

Column $\Rightarrow$ Separate data points

- MySQL
- Oracle
- MS SQL Server
- SQLite
- Postgres
- MariaDB

- Key-Value stores
- Document DB
- Wide-Column DB
- Graph DB

NO SQL

Key-Value Store

Data $\Rightarrow$ array of key-value pair
Key $\Rightarrow$ attribute

Value linked to

- Redis
- Voldemort
- Dynamo

Document DB

Data $\Rightarrow$ documents

$\Downarrow$ group into

Collections

Each doc can be different

- Couch DB
- Mongo DB

Wide-Column DB

Instead of tables, Columns, families.

Container for rows.

No need of knowing all columns upfront.

Each row $\Rightarrow$ diff. no. of columns

Analysis of large datasets.

- Cassandra
- HBase

Graph DB.

Data whose relations are best represented in Graphs.

- $\Rightarrow$ Nodes (Entities)
- $\Rightarrow$ Properties (Information of entities)
- $\Rightarrow$ Lines (Connection between entities)

- Neo 4J
- Infinite Graph.

→ High Level differences between SQL & NoSQL

PROPERTY	SQL	NOSQL
Storage	Tables (Row → Entity, Column → Data Point) e.g. Student (Branch, Id, Name)	Diff. data storage models. (Key, value, document, graph, columns)
Schema	Fixed schema (columns must be decided & chosen before data entry) can be altered ⇒ modify whole database (need to go offline)	Dynamic schemas. Columns addition on the fly. Not mandatory for each rows to contain data.
Querying	SQL.	UnQL (Unstructured Query Language). Queries focused on collection of documents. Diff. DB ⇒ diff UnQL.
Scalability	Vertically scalable. (+ horsepower of h/w) Expensive. Possible to scale accross multiple servers. ⇒ Challenging & time-consuming.	Horizontally scalable. Easy addition of servers. Hosted on cloud or cheap commodity h/w. → cost effective.
Reliability or Acid Compliancy	Acid Compliant ⇒ Data reliability. ⇒ Guarantee of transactions. ⇒ Still a better bet.	Sacrifice ACID compliance for scalability & performance. (Acid - Atomcity, Consistency, Isolation, Durability).



Reason to use SQL DB

1). You need to ensure ACID compliance :

ACID Compliance

⇒ Reduces anomalies

⇒ Protects Integrity of the database.

For many E-commerce & financial application

→ ACID Complaint DB is the first choice.

2). Your data is structured & unchanging.

If your business is not experiencing rapid growth or sudden changes.

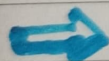
⇒ No requirements of more servers.

⇒ data is consistent.

then there's no reason to use System Design to support variety of data & high traffic.

ATUL KUMAR (LINKEDIN)

NOTES GALLERY (TELEGRAM)



Reason to use NOSQL DB

When all other components of system are fast querying & searching for data ⇒ bottleneck.

NOSQL prevent data from being bottleneck.

Big data ⇒ large success for NOSQL.

1). To store large volumes of data (little / no structure)

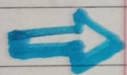
No limit on type of data.

Document DB ⇒ Stores all data in one place
(No need of type of data)



- 2). Using cloud & storage to the fullest.
Excellent cost saving solution. (Easy spread of data across multiple servers to scale up).
OR commodity h/w on site (affordable, smaller)
⇒ No headache of additional s/w.
& NOSQL DBs like Cassandra ⇒ designed to scale across multiple data centers out of the box.
- 3). Useful for rapid / agile development.
If you're making quick iterations on schema ⇒ SQL will slow you down.

ATUL KUMAR (LINKEDIN)
NOTES GALLERY (TELEGRAM)

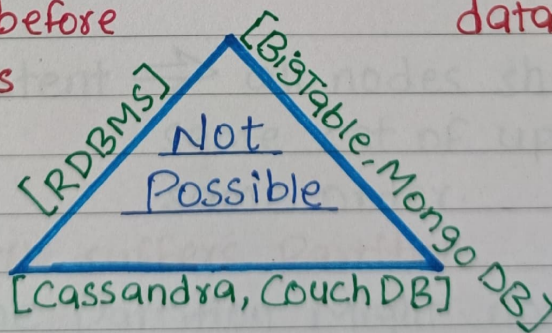


CAP Theorem.

Achieved by updating several nodes before allowing reads

Consistency

(All nodes see same data at same time)



Availability

Every request gets response (success / failure)

Partition Tolerance

System continues to work despite message loss / partial failure.

Achieved by replicating data across different Servers.

(can sustain any amount of network failure without resulting in failure of entire network.)

Data is sufficiently replicated across combination of nodes/networks to keep the system up.

It is impossible for a distributed system to simultaneously provide more than two of three of the above guarantees.

We cannot build a datastore which is :

- 1). Continually available
- 2). Sequentially Consistent
- 3). Partition failure tolerant.

ATUL KUMAR (LINKEDIN)
NOTES GALLERY (TELEGRAM)

Because,

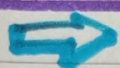
To be consistent $\Rightarrow$ all nodes should see the same set of updates in the same order.

But if network suffers Partition, update in one partition might not make it to other partitions.

$\hookrightarrow$ Client reads data from out-of-date partition After having read up-to-date partition.

Solution: Stop serving requests from out-of-date partition.

$\hookrightarrow$ Service is no longer 100% available.



Redundancy & Replication.

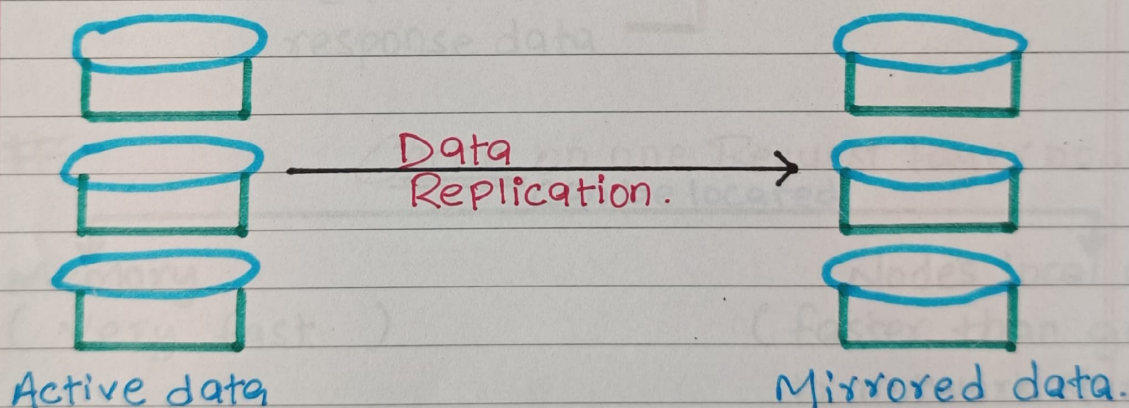
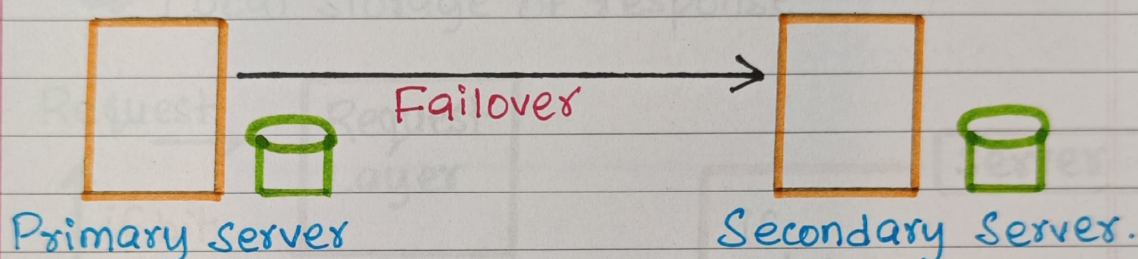
⇒ Duplication of critical data & Services.

↳ increasing reliability of system.

For critical services & data ⇒ ensure that multiple copies/versions are running simultaneously on different Servers / databases.

⇒ Secure against single node failures.

⇒ Provides backups if needed in crisis.



Service Redundancy: Shared - nothing architecture.

Every node ⇒ independent. No central service managing state.

More resilient to failures

New Servers addition without special conditions

Helps in scalability.

No single points of failure

Caching

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

Load balancing $\Rightarrow$ Scales horizontally

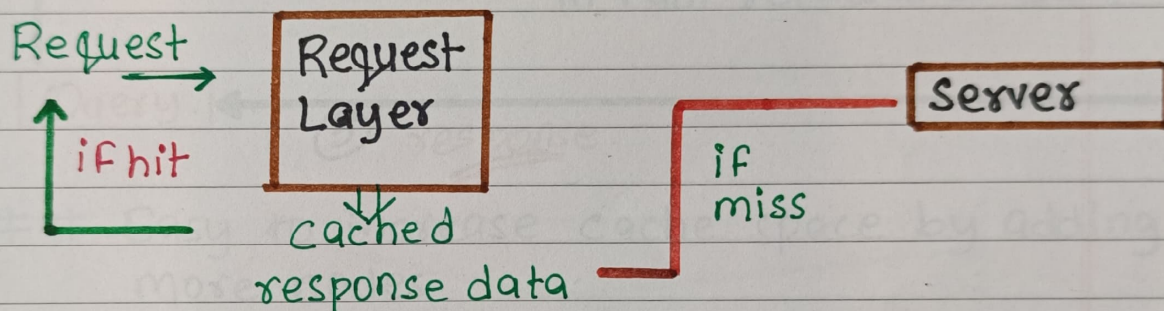
Caching : Locality of reference principle

$\hookrightarrow$ Used in almost every layer of computing.

1). Application Server cache:

Placing a cache directly on a request layer node.

$\hookrightarrow$ Local storage of response



#

Cache on one Request layer node.
Can be located

Memory
(very fast)

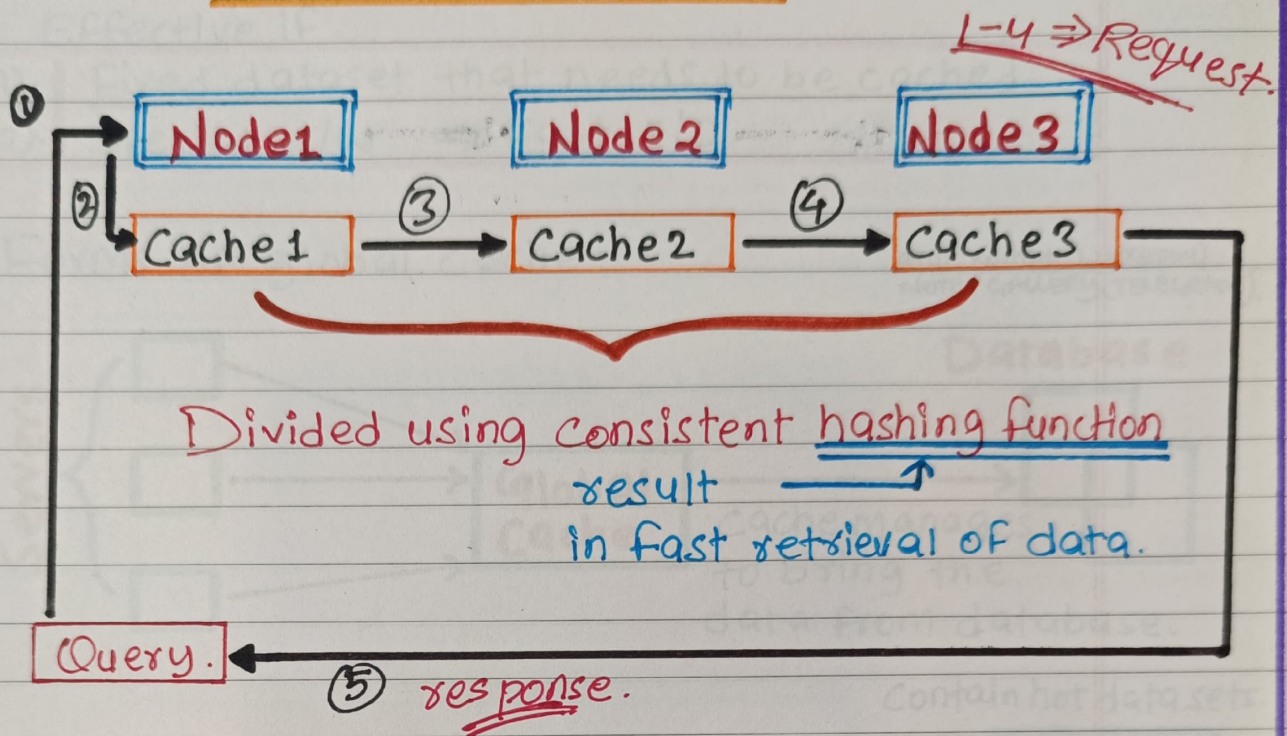
Node's local disk.
(faster than going to network storage)

: If LB distributes request randomly
Bottleneck $\hookrightarrow$ Same request $\Rightarrow$ different nodes
More cache miss

Can be
overcome
by.

- | | |
|-----|--------------------|
| 1). | Global caches |
| 2). | Distributed caches |

Distributed cache



Easy to increase cache space by adding more nodes.

Disadvantages: Resolving a missing node. starting multiple copies of data on different nodes. $\leftarrow$ Can be handled by.
 $\Rightarrow$ Were making it more complicated.

Even if node disappears $\Rightarrow$ request can pull data from origin.

ATUL KUMAR (LINKEDIN),
NOTES GALLERY (TELEGRAM).

Global cache.

Single cache space for all the nodes.

$\rightarrow$ Adding a cache servers / File store (faster than original store)

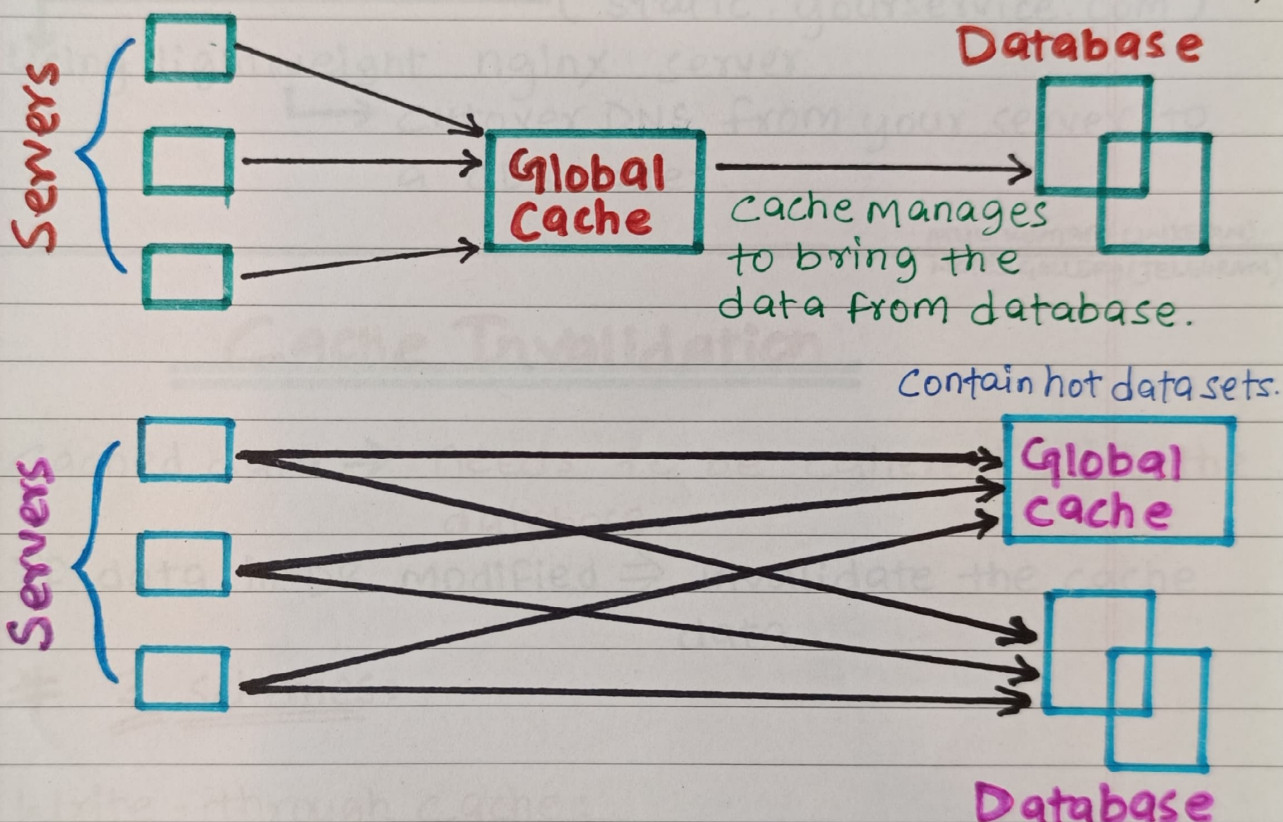
Difficult to manage if no clients / request increases.

Effective if

- 1). Fixed dataset that needs to be cached.
- 2). Special H/W $\Rightarrow$ Fast I/O.

Forms of global cache:

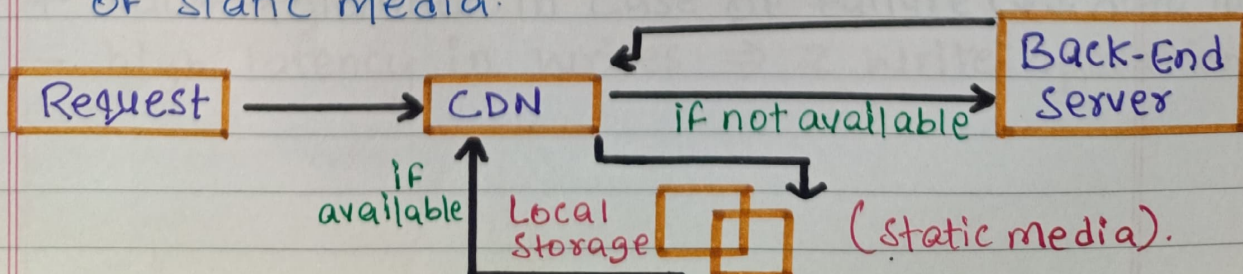
ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).



Application I logic understands the strategy / hot spots better than cache.

CDN: Content Distribution network.

Cache store for sites that serves large amount of static media.



If the site isn't large enough to have its own CDN.

For better & easy future transition.

Serve static media using separate subdomain.

(static.yourservice.com)

Using lightweight nginx server

↳ Cutover DNS from your server to a CDN later.

ATUL KUMAR (LINKEDIN) ·
NOTES GALLERY (TELEGRAM)



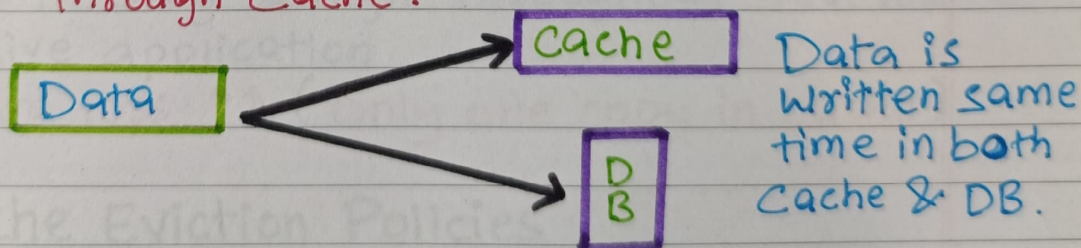
Cache Invalidation

Cached data ⇒ needs to be coherent with the database.

If data in DB modified ⇒ invalidate the cache data.

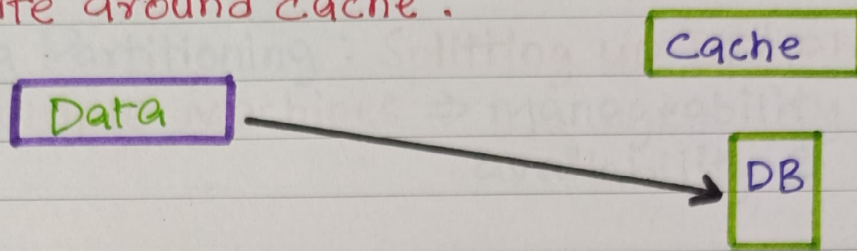
3 schemes:

1). Write-through cache:



- + Complete data consistency (cache = DB)
- + Fault tolerance in case of failure (↓↓ data loss)
- high latency in writes ⇒ 2 write operations.

2). Write around cache :

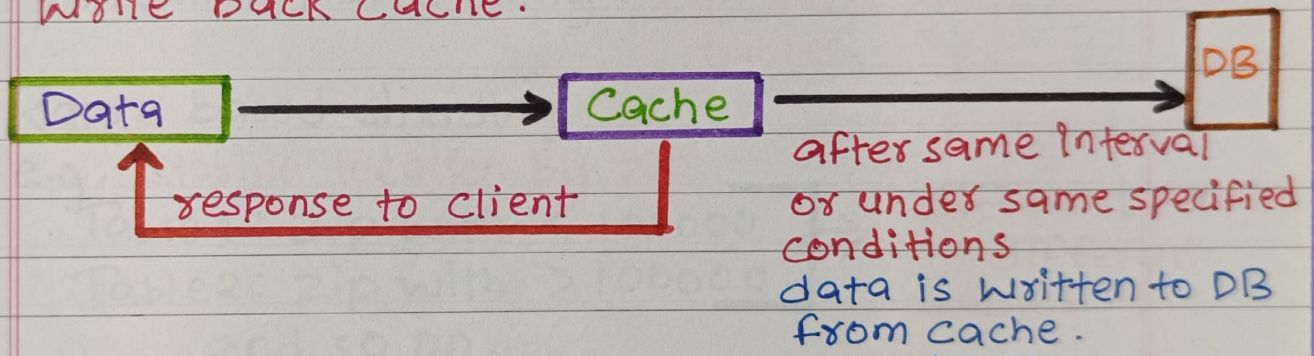


+ No cache for writes.

- read request for newly written data $\Rightarrow$ Miss
higher latency.

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

3). Write back cache :



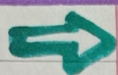
+ low latency & high throughput for write-intensive application.

- Data loss $\uparrow\uparrow$ (only one copy in cache)

#

Cache Eviction Policies

1).	FIFO
2).	LIFO or FILO
3).	LRU
4).	MRU
5).	LFU
6).	Random Replacement



Sharding || Data Partitioning

Data Partitioning : Splitting up DB/table across multiple machines $\Rightarrow$ manageability, performance, availability & LB.

★★ After a certain scale point, it is cheaper and more feasible to scale horizontally by adding more machines instead of vertical scaling by adding buffer services.

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

Methods of Partitioning :

1). Horizontal Partitioning : Different rows into diff. tables.

Range based sharding

e.g. storing location by zip

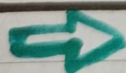
Table 1: Zip with < 100000

Table 2: Zip with > 100000
and so on.

$\Rightarrow$ different ranges in different tables.

★★ Cons: if the value of the range not chosen carefully $\Rightarrow$ leads to unbalanced servers.

e.g. Table 1 can have more data than table 2.



Vertical Partitioning.

Feature wise distribution of data.

$\hookrightarrow$ in different servers.

e.g. Instagram

- $\rightarrow$ DB Server 1: user info
- $\rightarrow$ DB Server 2: followers
- $\rightarrow$ DB Server 3: Photos

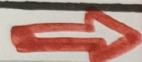
★★
★★
☹☹

Straightforward to implement
low impact on app.

if app → additional growth.

need to partition feature specific DB across various servers.

(e.g. it would not be possible for a single server to handle all metadata queries for 10 million photos by 140 millions users.



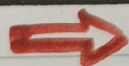
Directory based Partitioning

⇒ A loosely coupled approach to work around issues mentioned in above two partitionings.

★★ Create lookup services ⇒ current partitioning scheme & abstracts it away from the DB access code.

Mapping (tuple key → DB server)

Easy to add DB servers or change partitioning scheme.



Partitioning Criteria

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

i). Key or Hash based partitioning :

Key attr. of the data. → Hash function → Partition number

Effectively fixes the total number of servers / partition.



So if we add new server / partition ↴

Change in hash function.
downtime because of ↴
redistribution.

Solution: Consistent Hashing.

2). List Partitioning: Each partition is assigned a list of values.

new record. → Lookup for key. → Store the record (Partition based on the key)

3). Round Robin Partitioning:
uniform data distribution.

with 'n' partitions

⇒ the 'i' tuple is assigned to partition $(i \bmod n)$

4). Composite Partitioning:

Combination of above partitioning schemes
Hashing + List ⇒ Consistent Hashing.



Hash reduces the key space to a size that can be listed.

Common Problems of Sharding:

Sharded DB: Extra constraints on the different operations.



Operations across multiple tables or multiple rows in the same table.

no longer running in single server.

1). Joins & Denormalization:

Joins on tables on single server ⇒ straight forward.

- ★ not possible to perform joins on standard tables.
 - ↳ Less efficient (data need to be compiled from multiple servers)

Workaround ⇒ Denormalize the DB.

so that the queries that previously reqd. joins can be performed from a single table.

↳ Cons: Perils of denormalization.

- ↳ data inconsistency.

2). Referential integrity: foreign keys on Sharded DB.

↳ difficult.

- ★ Most of the RDBMS does not support foreign keys on sharded DB.

If appⁿ demands referential integrity on sharded DB.

- ↳ enforce it in appⁿ code (SQL jobs to clean up dangling reference).

3). Rebalancing:

Reasons to change sharding scheme:

- | | |
|-----|---|
| a). | non-uniform distribution (data wise) |
| b). | non-uniform load balancing (request wise) |

Workaround:

- 1). add new DB.
- 2). rebalance

↳ Change in partitioning scheme

↳ data movement.

↳ downtime.

We can use directly - based partitioning.

↳ highly complex

↳ Single point of failure.

(lookup service/table)



Indexes

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

⇒ Well known because of database.

⇒ Improves speed of retrieval.

— Increased storage overhead.

— Slower writes.

↳ Write the data.

↳ Update the index.

⇒ Can be created using one or more columns.

★ Rapid random lookups.

& efficient access of ordered records.

Data Structure.

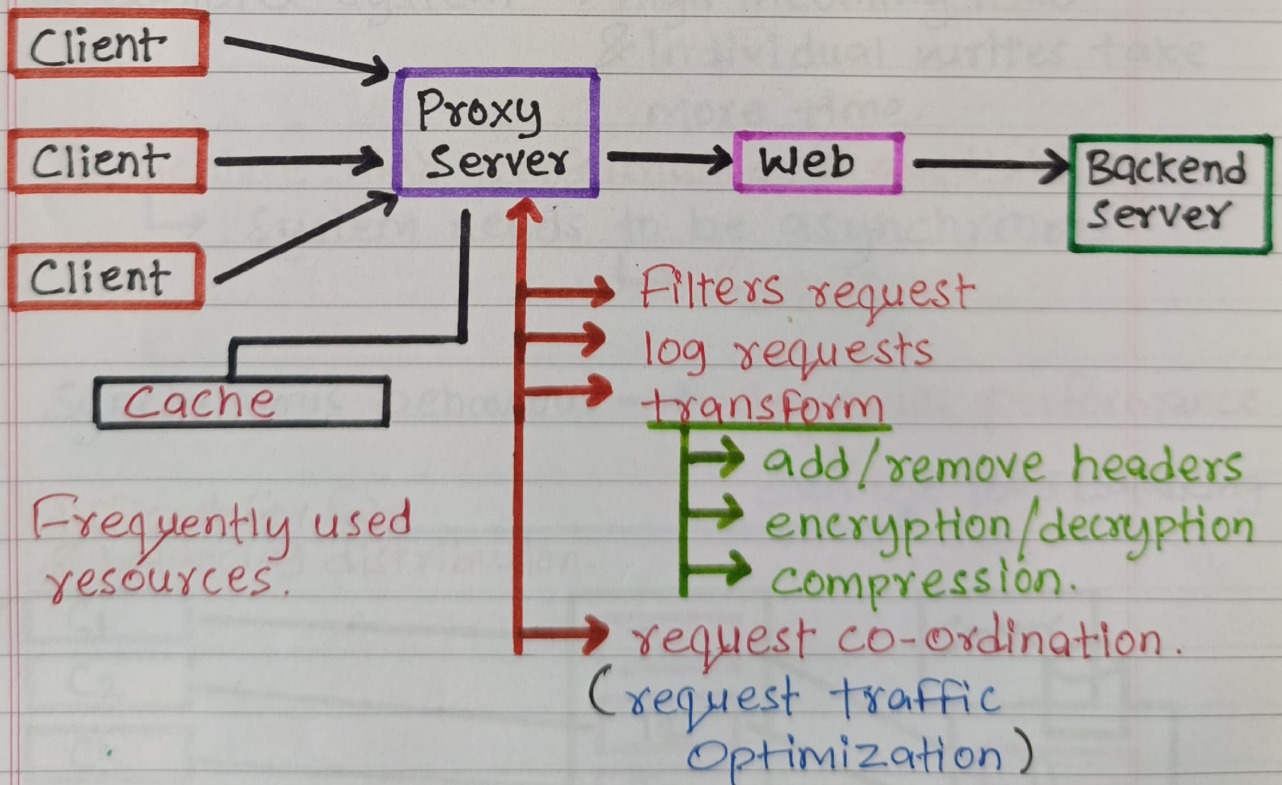
Column → Pointer to whole row.

- Create different views of the same data.
 ↳ Very good for filtering / sorting of large data sets.
 ↳ no need to create additional copies.

Using for datasets (TB in size) & small payload (KB).
 Spread over several physical devices. → We need some way to find the correct physical location. i.e.

Indexes

Proxies → useful under high load situations.
 → if we have limited caching.
 ↳ batches several requests into one.



We can also use

Spatial locality

↳ Collapsing request
for data that is
Spatially close.

← Collapse same data access
request into one.

⇒ Collapsed forwarding

Minimize reads from
Origin.

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

Queues

⇒ Effectively manages requests in large-scale distributed system.

→ In small system → writes are fast

→ In complex system → high incoming load
& individual writes take
more time.

★ To achieve high performance & availability.

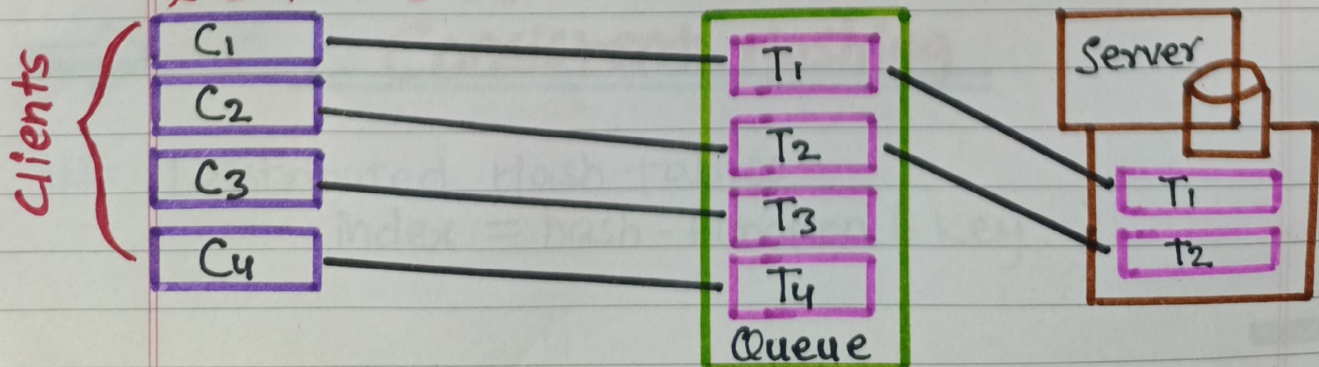
↳ System needs to be asynchronous

↳ Queue

Synchronous behaviour → degrades performance.

↓
can use load balancing.

difficult for fair
& balancing distribution.



Queues: asynchronous communication protocol.

↳ Client sends task

↳ get ACK from queue (receipt)

↳ Server as references for the results in future.

↳ Client continues its work.

Limit on the size of request
& number of requests in queue

Queue: Provides fault tolerance.

↳ Protection from service outage / failure.

↳ highly request

↳ retry failure service request

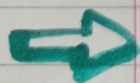
↳ Enforce Quality of services guarantee
(Does NOT expose clients to outage)

Queues: distributed communication.

↳ Open Source implementation.

↳ RabbitMQ, ZeroMQ, activeMQ, BeanstalkD.

ATUL KUMAR (LINKEDIN)
NOTES GALLERY (TELEGRAM)



Consistent Hashing

Distributed Hash tables

index = hash-function (key)



→
Suppose we're designing distributed caching system with n cache services

↳ hash-function $\Rightarrow (Key \% n)$

Drawbacks:

1). NOT horizontally scalable

↳ addition of new server results in

↳ needs to change all existing mapping.

(downtime of system)

2). NOT load balanced

(because of non-uniform distribution of data)

↓
Some caches: hot & saturated
Other caches: idle & empty

• How to tackle above problems?

→ Consistent Hashing

• What is consistent hashing?

→ Very useful strategy for distributed caching & DHTs.

→ Minimize reorganization in scaling up/down.

→ Only $\boxed{K/n}$ Keys need to be remapped.

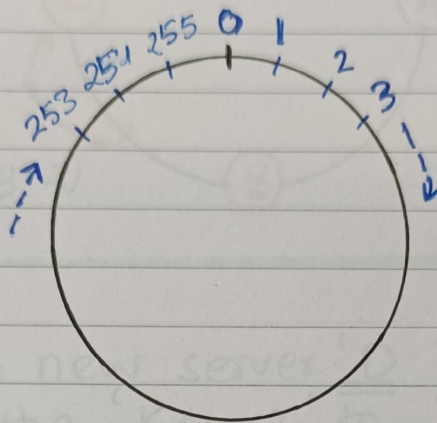
K = total numbers of Keys

n = number of servers.

• How it works?

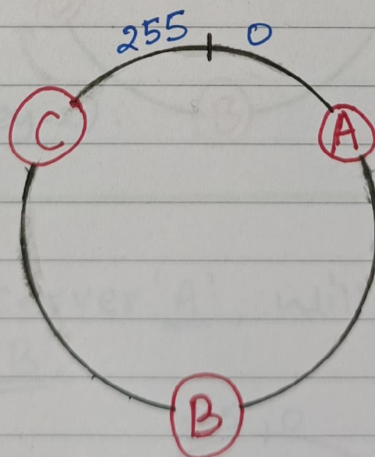
→ Typical hash function suppose outputs in $[0, 256]$

In consistent hashing,
imagine all of these integers are placed
on a ring.



& we have 3 servers : A, B & C.

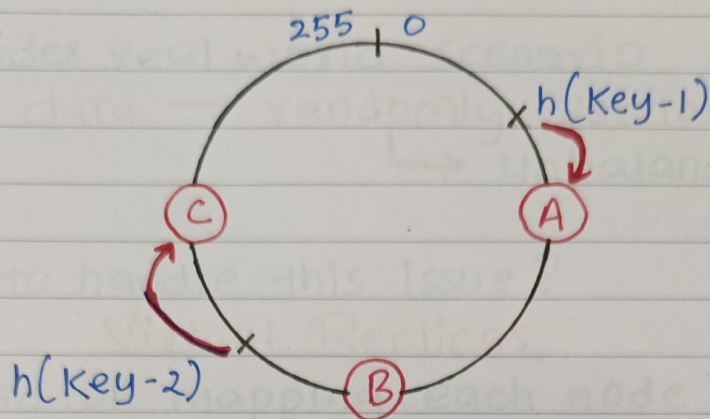
- 1). Given a list of servers, hash them to integers in the range.



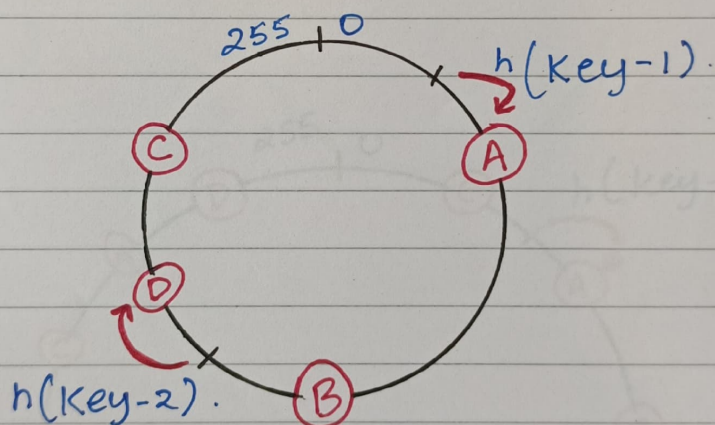
- 2). Map key to a server:

- Hash it to single integer
- Move clockwise until you find server
- Map key to that server.

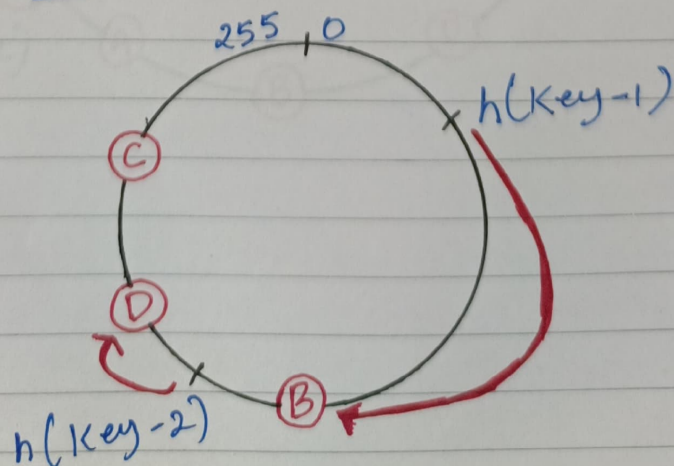




Adding a new server 'D', will result in moving the 'Key-2' to 'D'.



Removing server 'A', will result in moving the 'Key-1' to 'B'.



Consider real world scenario

data $\rightarrow$ randomly distributed

$\rightarrow$ unbalanced caches.

How to handle this issue?

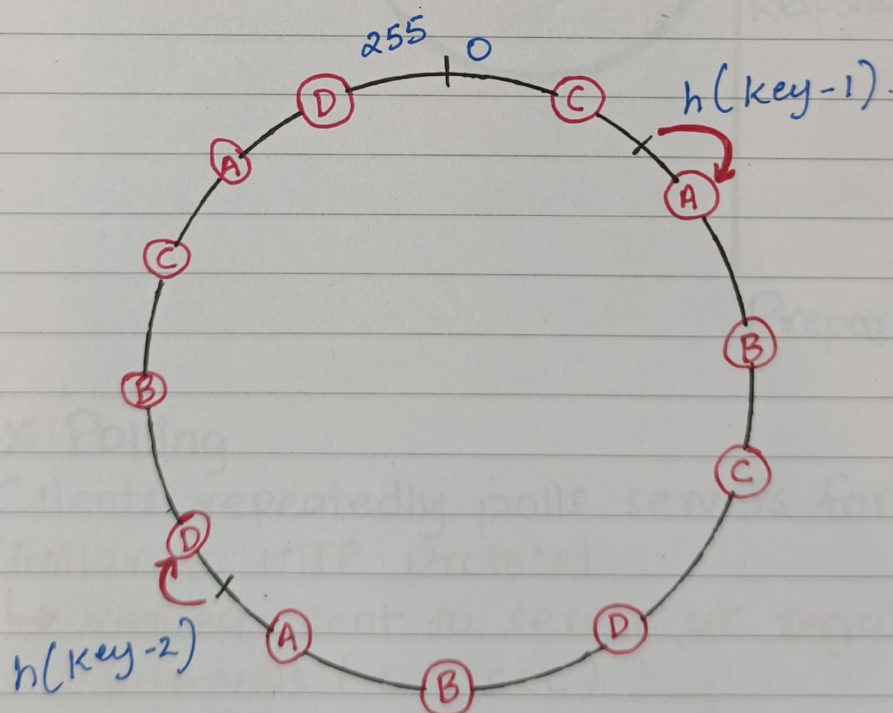
Virtual Replicas

$\Rightarrow$ Instead of mapping each node to a single point we map it to Multiple points.

$\rightarrow$ (More number of replicas

$\rightarrow$ More equal distribution

$\rightarrow$ good load balancing)

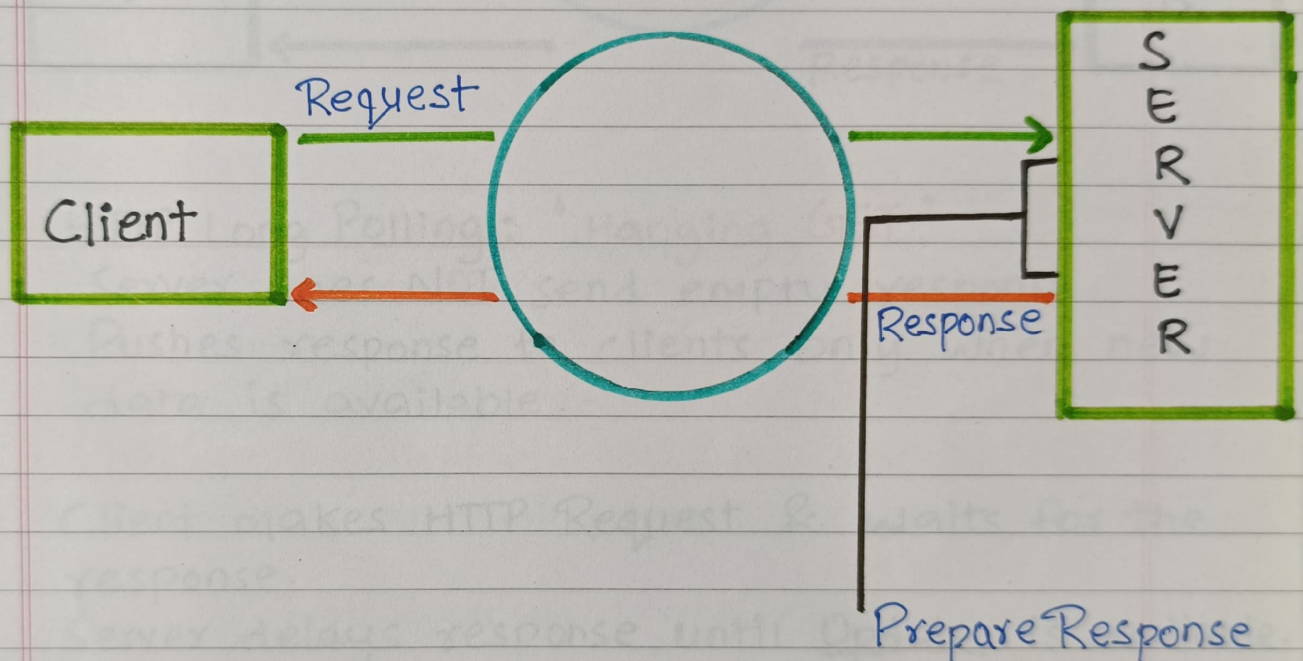


→ Long-Polling vs WebSockets vs Server-Sent Events.

↳ Client - Server Communication Protocols.

HTTP Protocol :

ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).



AJAX Polling

Clients repeatedly polls servers for data
Similar to HTTP protocol

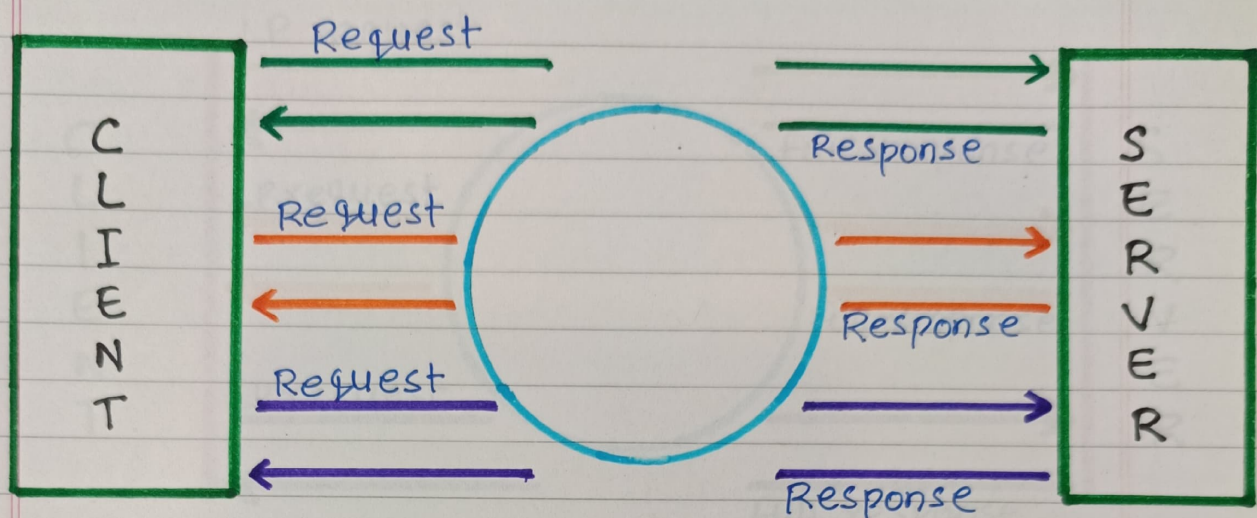
↳ request sent to server at regular intervals (0.5 sec)

Drawbacks:

Client keeps asking the server new data

↳ Lot of response are 'empty'

↳ HTTP Overhead.



HTTP Long Polling: 'Hanging GET'

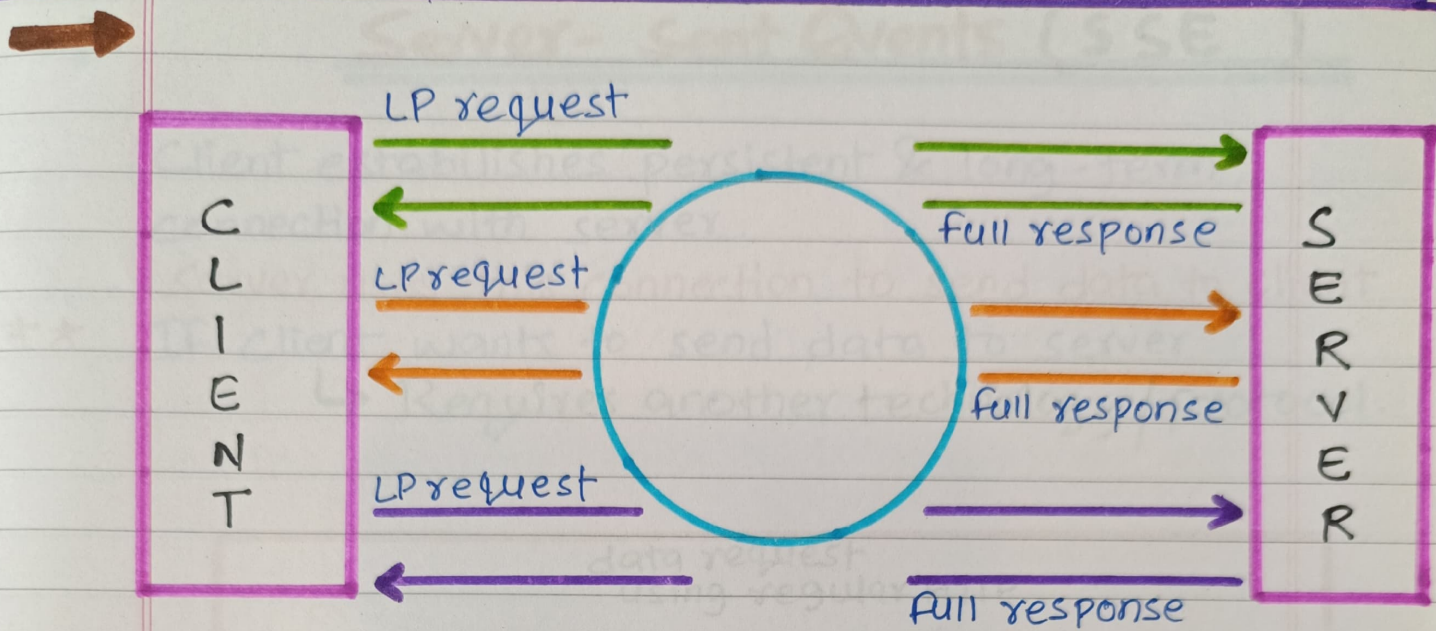
Server does NOT send empty response.

Pushes response to clients only when new data is available.

- 1). Client makes HTTP Request & waits for the response.
- 2). Server delays response until Update is available. or until timeout occurs.
- 3). When Update → Server sends full response.
- 4). Client sends new long-poll request.

- | |
|--|
| a). immediately after receiving response.
b). after a pause to allow acceptable latency period. |
|--|

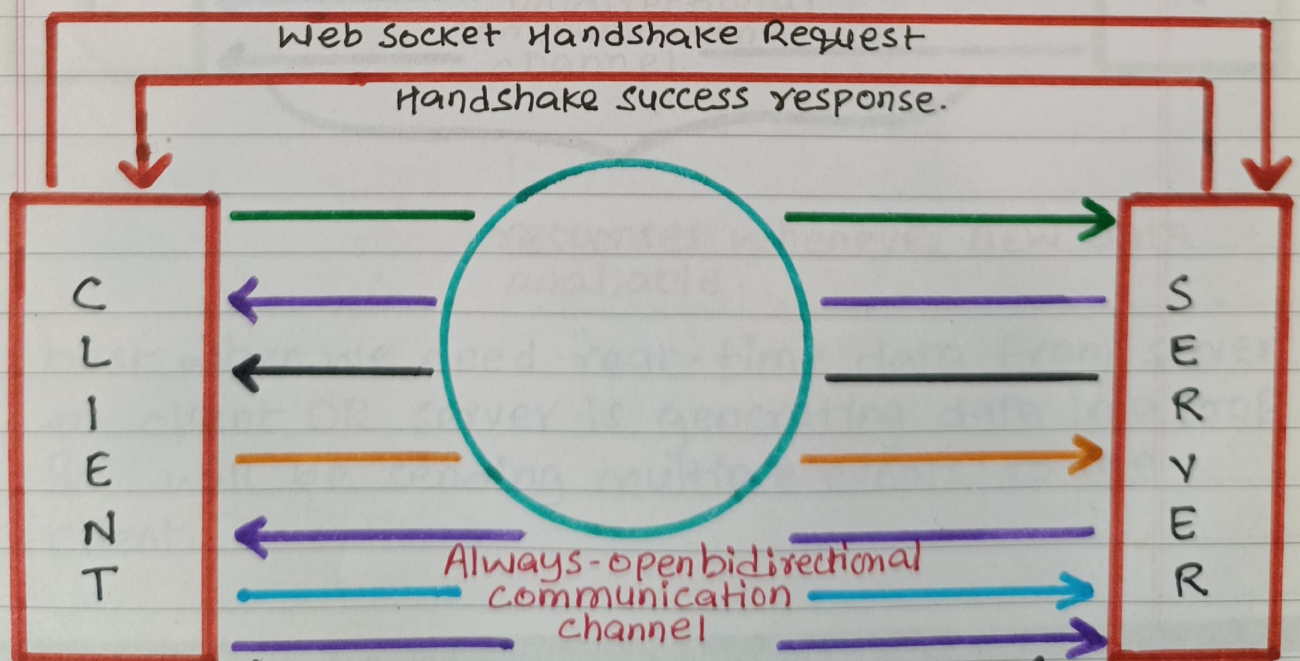
- 5). Each request has timeout.
Client needs to reconnect periodically due to timeouts.



ATUL KUMAR (LINKEDIN).
NOTES GALLERY (TELEGRAM).

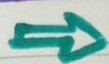
Web Sockets

- Full duplex communication channel over single TCP connection.
- Provides 'Persistent communication' (client & server can send data at anytime)
- bidirectional communication is always open channel.



→ Lower overheads.

→ Real time data transfer.



Server-Sent Events (SSE)

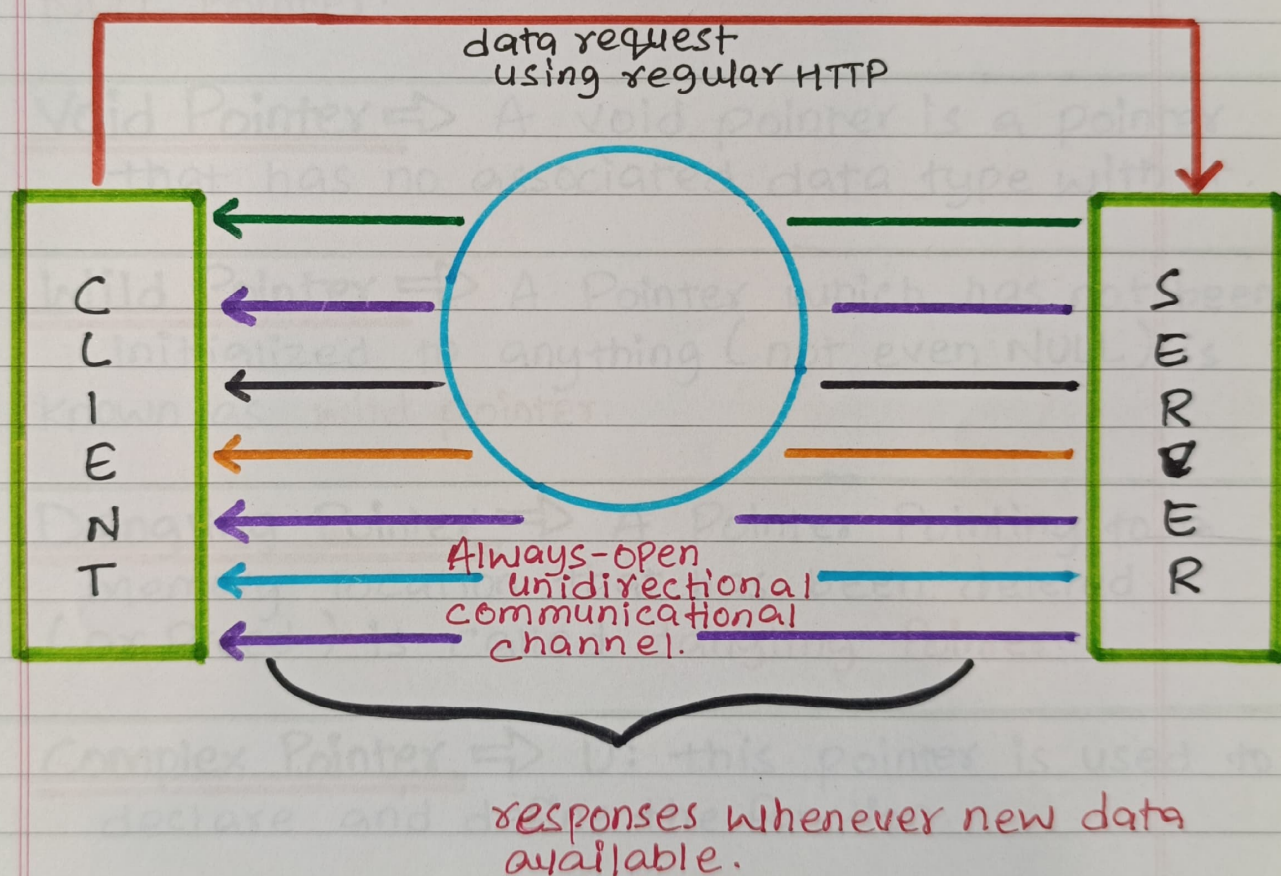
Client establishes persistent & long-term connection with server.

Server uses this connection to send data to client

★★

If client wants to send data to server

↳ Requires another technology/protocol.



→ best when we need real-time data from server to client **OR** server is generating data in a loop & will be sending multiple events to the client.